

NoLie — Technical Report for Mentor Meeting

1. Project Overview

NoLie is a Chrome extension that provides AI-powered content credibility assessment and fact verification. It analyzes news articles, YouTube videos, and any audio playing in a browser tab, extracting factual claims, verifying them against known information, scoring source reliability, and presenting results in a real-time dashboard.

2. Complete Feature List (What Was Implemented)

2.1 Article Scanning

- One-click full page analysis of any news article
- Automatic extraction of article text, images, embedded videos, and metadata
- Batch extraction of all verifiable factual claims
- Individual verification of each claim with full article context
- Overall credibility score (0-100) based on weighted verdict analysis
- Credibility heatmap overlay directly on the article (green/yellow/red highlights)
- Source credibility badge (MBFC dataset — 3,920 domains)

2.2 YouTube Video Fact-Checking

- Automatic transcript extraction from YouTube's built-in captions
- Works with auto-generated captions in any language
- Auto-opens the transcript panel programmatically
- Full transcript runs through claim extraction and verification pipeline

2.3 Real-Time Live Fact-Checking (YouTube)

- Caption polling via DOM observation (1-second interval)
- Rolling window of caption text accumulated
- Claims extracted and verified as video plays
- Results appear in side panel in real-time

2.4 Real-Time Live Fact-Checking (Any Audio — Deepgram)

- Tab audio capture via Chrome's tabCapture API
- Real-time transcription via Deepgram Nova-2 WebSocket
- Audio passthrough so user still hears the content
- Transcript displayed live in the panel
- Claims extracted from accumulated transcript
- Works on Instagram, podcasts, live streams, Twitter Spaces — any tab with audio

2.5 Manual Fact-Checking

- Right-click context menu: "Fact Check This"
- Select any text on any page and verify it
- Side panel opens automatically with results

2.6 Image Analysis

- AI-generated image detection
- Image manipulation detection
- Misleading chart/graph detection

- Caption mismatch detection
- Powered by Google Gemini multimodal

2.7 Source Credibility Scoring

- MBFC (Media Bias/Fact Check) dataset bundled offline
- 3,920 news domains with bias and factual reporting ratings
- Domain lookup with subdomain fallback
- Adjusts overall credibility score based on source reliability

2.8 History and Export

- All scans saved with full results and transcript
- Click any history item to view past analysis
- Clear history button
- Export as downloadable HTML report with citations
- Live sessions saved with full transcript text

2.9 Customizable Verification

- User-editable fact-check prompt
- Supports `[[claim]]` and `[[context]]` placeholders
- Reset to default button
- Auto-saves on edit

2.10 UI/UX

- Dark Notion-style theme throughout (side panel, popup, options)
- Chrome Side Panel API for persistent results display
- Tabbed interface (Results, Live, History)
- Loading animations with progress bar
- Color-coded verdicts and confidence levels
- Clickable source citations (Google search for named sources)

3. Tech Stack — Detailed Breakdown

3.1 Chrome Extension (Manifest V3)

What it is: Chrome's latest extension format. Replaces background pages with service workers, enforces stricter security, uses declarative APIs.

Where we use it: The entire project is a Chrome MV3 extension. The `manifest.json` declares permissions, content scripts, service worker, side panel, popup, and offscreen document.

Key permissions used:

- `storage` — persisting API keys, settings, scan history
- `activeTab` — accessing the current tab's URL and content
- `scripting` — injecting content scripts dynamically
- `contextMenus` — right-click "Fact Check This" menu item
- `sidePanel` — Chrome's side panel API for results display
- `tabCapture` — capturing audio from browser tabs
- `offscreen` — creating hidden documents for audio processing

3.2 Vite + @crxjs/vite-plugin

What it is: Vite is a modern build tool. `@crxjs/vite-plugin` is a plugin that understands Chrome extension manifests and bundles everything correctly.

Where we use it: Build tooling. Vite reads our `manifest.json`, bundles all JS files with tree-shaking, processes CSS, copies static assets, and outputs a ready-to-load `dist/` folder.

Why: Without a bundler, we'd need to manage module imports manually. Vite lets us use ES modules (`import/export`) everywhere and compiles them into extension-compatible bundles.

3.3 Groq API (Llama 3.3 70B)

What it is: Groq provides ultra-fast LLM inference on custom LPU hardware. We use their API to run Llama 3.3 70B (a 70 billion parameter open-source model).

Where we use it:

- `src/lib/groq.js` — all text-based AI calls
- Claim extraction: given article text, extract verifiable factual claims as structured JSON
- Claim verification: given a claim + article context, return verdict/confidence/explanation/sources
- Live claim extraction: stricter prompt for real-time mode (filters out opinions, greetings, filler)

How the API call works:

```
POST https://api.groq.com/openai/v1/chat/completions
Headers: Authorization: Bearer <API_KEY>
Body: {
  model: "llama-3.3-70b-versatile",
  messages: [{ role: "system", content: "..."}, { role: "user", content: "..."}],
  temperature: 0,
  response_format: { type: "json_object" }
}
```

Why Groq over other providers:

- Free tier: 14,400 requests/day, 30 RPM
- Extremely fast inference (~500 tokens/sec)
- Structured JSON output mode
- No credit card required
- Llama 3.3 70B is excellent for structured extraction tasks

3.4 Google Gemini API (3.6 Flash)

What it is: Google's multimodal AI model that can process both text and images natively.

Where we use it:

- `src/lib/gemini.js` — image analysis and video thumbnail analysis
- Sends image URLs (fetched and converted to base64) along with text prompts
- Detects: AI-generated content, manipulated images, misleading charts, caption mismatches

How the multimodal call works:

```
POST https://generativelanguage.googleapis.com/v1beta/models/gemini-3.6-flash:generateContent?
key=<KEY>
Body: {
  contents: [{
```

```

    parts: [
      { text: "Analyze this image..." },
      { inlineData: { mimeType: "image/jpeg", data: "<base64>" } }
    ]
  }]
}

```

Why Gemini: Only major free-tier provider with native multimodal (image) support.

3.5 Deepgram API (Nova-2)

What it is: Real-time speech-to-text API using their Nova-2 model. Connected via WebSocket for streaming transcription.

Where we use it:

- `src/offscreen/offscreen.js` — WebSocket connection to Deepgram
- Receives raw PCM audio from tab capture
- Streams audio chunks to Deepgram in real-time
- Receives transcribed text back via the same WebSocket

How the WebSocket connection works:

```

socket = new WebSocket(
  'wss://api.deepgram.com/v1/listen?
encoding=linear16&sample_rate=16000&channels=1&model=nova-
2&language=en&punctuate=true&interim_results=true&utterance_end_ms=2500&smart_format=true&vad_e
  ['token', DEEPGRAM_API_KEY]
);

```

WebSocket message flow:

1. Client sends: raw Int16 PCM audio chunks (4096 samples at a time)
2. Server sends back: JSON with `channel.alternatives[0].transcript`, `is_final`, `speech_final`
3. `UtteranceEnd` event signals end of a complete thought/sentence
4. Client accumulates partial results until `speech_final` is true

Why WebSocket: Real-time bidirectional communication. HTTP REST would require sending entire audio files — WebSocket allows streaming as audio is captured.

3.6 MBFC Dataset

What it is: Media Bias/Fact Check is an independent fact-checking organization that rates news sources. We bundled their dataset as a 180KB JSON file.

Where we use it:

- `src/data/mbfc.json` — 3,920 domain entries with bias and factual reporting ratings
- `src/lib/mbfc.js` — lookup function with domain normalization and subdomain fallback

How it works:

```

lookupDomain("www.bbc.com")
// Strips "www.", looks up "bbc.com"

```

```
// Returns: { rating: "HIGH", bias: "CENTER", domain: "bbc.com" }
```

Why bundled offline: No API call needed, instant lookup, works without internet, no rate limits.

3.7 Vercel Serverless Functions (Backend)

What it is: Vercel hosts serverless functions that run on-demand. We use one to securely provide the Deepgram API key.

Where we use it:

- `nolie-backend/api/deepgram-token.js` — single endpoint
- Extension's offscreen document calls `POST /api/deepgram-token`
- Backend returns the Deepgram key stored as an environment variable

Why a backend: The Deepgram API key can't be stored in extension client code (anyone could inspect it). The backend keeps it server-side. This is the same architecture pattern used in production applications.

3.8 Chrome Side Panel API

What it is: Chrome's built-in side panel that slides in from the right side of the browser. Persists while user navigates.

Where we use it:

- `src/sidepanel/sidepanel.html` — the main results UI
- Opened via `chrome.sidePanel.open({ tabId })`
- Receives messages from service worker to display results, progress, errors, live claims

Why Side Panel over popup: Popups close when you click away. The side panel stays open while you browse, which is essential for displaying scan results and live fact-checking.

3.9 Chrome Offscreen Document

What it is: A hidden HTML page that runs in the background. Required for operations that need DOM APIs (like audio processing) which aren't available in service workers.

Where we use it:

- `src/offscreen/offscreen.html` + `offscreen.js`
- Captures tab audio via `navigator.mediaDevices.getUserMedia` with `chromeMediaSource: 'tab'`
- Creates `AudioContext` for processing audio at 16kHz
- Maintains `WebSocket` connection to Deepgram
- Streams audio and receives transcription

Why offscreen: Service workers can't access Web Audio API or `MediaStream`. The offscreen document provides a DOM context for audio processing without being visible to the user.

3.10 Web Audio API

What it is: Browser API for processing and analyzing audio in real-time.

Where we use it:

- `src/offscreen/offscreen.js` — `AudioContext` and `ScriptProcessor`
- Creates a 16kHz audio context from the tab's media stream
- `ScriptProcessor` node intercepts audio frames (4096 samples per frame)
- Converts `Float32` samples to `Int16` PCM format (Deepgram's required format)

- Pipes audio to both Deepgram (for transcription) and speakers (for user to hear)

Audio processing pipeline:

```
Tab Audio Stream → MediaStreamSource → ScriptProcessor (Float32→Int16) → WebSocket to Deepgram
  ↳ AudioContext.destination (user hears audio)
```

3.11 Content Scripts

What it is: JavaScript that runs in the context of web pages the user visits.

Where we use it:

- `src/content/content.js` — extracts article text, images, metadata from any page
- `src/content/youtube.js` — YouTube-specific transcript extraction
- `src/content/live-factcheck.js` — caption polling for YouTube live mode
- `src/content/heatmap.css` — inline highlight styles

How content extraction works:

1. Finds article container (checks for `<article>`, `[role="main"]`, `.post-content`, etc.)
2. Extracts all `<p>`, `<h1-h6>`, `<blockquote>`, `<li>` text
3. Finds images (filters out tiny icons, avatars, logos)
4. Finds YouTube embeds (extracts video ID for thumbnail URL)
5. Gets metadata: domain, author (from meta tags), publish date

3.12 MutationObserver / Polling

What it is: Browser API for watching DOM changes. We use polling (`setInterval`) for YouTube captions.

Where we use it:

- `src/content/live-factcheck.js` — polls `.ytp-caption-segment` elements every 1 second
- When caption text changes, it's added to the buffer
- After 6 segments or 200+ characters, sends batch for verification

Why polling over MutationObserver: YouTube's caption elements are dynamically created/destroyed.

MutationObserver was unreliable because the container element doesn't exist when captions aren't showing. Polling every 1 second is simple and guaranteed to catch all caption changes.

4. Architecture Diagrams

4.1 Article Scanning Flow

```
User clicks "Scan This Page"
↓
Popup → chrome.sidePanel.open() → service-worker receives SCAN_PAGE
↓
service-worker → chrome.tabs.sendMessage → content.js extracts page content
↓
content.js → returns { text, images, videos, metadata }
↓
service-worker → Groq API: extract claims (structured JSON)
↓
```

```
service-worker → Groq API: verify each claim (with full article context)
↓
service-worker → Gemini API: analyze images (optional, multimodal)
↓
service-worker → MBFC lookup: check domain credibility
↓
service-worker → calculate score → broadcast SCAN_COMPLETE
↓
sidepanel.js → renders results + heatmap applied to page
```

4.2 YouTube Transcript Flow

```
User clicks "Scan This Page" on YouTube
↓
service-worker detects youtube.com/watch
↓
service-worker → chrome.scripting.executeScript (world: MAIN)
  → reads ytInitialPlayerResponse for caption track URL
↓
service-worker → auto-opens transcript panel via DOM manipulation
↓
service-worker → chrome.scripting.executeScript
  → reads transcript panel innerText
↓
Transcript text → same pipeline as article scanning
```

4.3 Live Mode (YouTube — Caption Polling)

```
User clicks "Start Live Fact-Check" on YouTube
↓
service-worker → content.js: START_LIVE
↓
live-factcheck.js → setInterval every 1000ms
  → queries .ytp-caption-segment elements
  → if new text found, adds to buffer
↓
Every 6 segments or 200+ chars:
  → sends LIVE_BATCH to service-worker
↓
service-worker → Groq: extract claims (strict prompt)
  → Groq: verify claims (with rolling context)
  → broadcast LIVE_CLAIMS to sidepanel
↓
sidepanel → appends claim cards in real-time
```

4.4 Live Mode (Deepgram — Any Audio)

```
User clicks "Start Live Fact-Check" on non-YouTube page
↓
service-worker → chrome.tabCapture.getMediaStreamId(tabId)
```

```
↓
service-worker → chrome.offscreen.createDocument('offscreen.html')
↓
service-worker → sendMessage: START_CAPTURE { streamId }
↓
offscreen.js → fetch('nolie-backend.vercel.app/api/deepgram-token')
→ receives Deepgram API key
↓
offscreen.js → navigator.mediaDevices.getUserMedia({ chromeMediaSource: 'tab' })
→ captures tab audio stream
↓
offscreen.js → new AudioContext({ sampleRate: 16000 })
→ source.connect(destination) [user hears audio]
→ ScriptProcessor converts Float32 → Int16
↓
offscreen.js → new WebSocket('wss://api.deepgram.com/v1/listen', ['token', key])
→ sends Int16 PCM chunks via WebSocket
↓
Deepgram → returns transcription JSON via WebSocket
↓
offscreen.js → chrome.runtime.sendMessage: TRANSCRIPT_FINAL
↓
service-worker → handleTranscriptSentence()
→ accumulates text
→ when 200+ chars: sends to Groq for claim extraction
→ verifies claims
→ broadcasts LIVE_CLAIMS
↓
sidepanel → displays transcript + claim cards in real-time
```

5. Problems Faced and How They Were Solved

Problem 1: Gemini API Quota Exhaustion

Issue: Gemini free tier only allows ~20 grounded requests/day. During development we burned through the quota quickly, making the extension unusable. **Solution:** Switched to a hybrid architecture — Groq (free, 14,400 req/day) handles all text operations (extraction + verification), Gemini only used for image analysis (optional, toggleable in settings). This reduced Gemini usage from ~15 calls/scan to 0-5 calls/scan.

Problem 2: Claims Verified Without Context

Issue: When verifying "The incident happened 1.5km inside the tunnel", the AI had no idea WHICH tunnel or WHERE, and marked everything as UNVERIFIABLE. **Solution:** Modified the verification pipeline to pass the full article text as context alongside each claim. The prompt now says "Article context: [full text]. Verify this specific claim from the article above." This dramatically improved accuracy — claims went from mostly UNVERIFIABLE to correctly assessed TRUE/FALSE.

Problem 3: YouTube Transcript Extraction

Issue: YouTube doesn't provide a public API for transcripts. The timedtext URLs expire within seconds. The `ytInitialPlayerResponse` global variable isn't accessible from content scripts (isolated world). **Solution:** Used `chrome.scripting.executeScript` with `world: 'MAIN'` to access YouTube's page-context JavaScript globals.

Fell back to reading the transcript panel DOM directly (auto-opening it programmatically). The final working approach: auto-open transcript panel → read `innerText` from the engagement panel → filter out timestamps and UI text.

Problem 4: MutationObserver Not Detecting YouTube Captions

Issue: YouTube's caption elements (`.ytp-caption-segment`) are dynamically created/destroyed. MutationObserver failed because it couldn't find the container at startup (captions only appear when text is shown). **Solution:** Replaced MutationObserver with simple 1-second polling (`setInterval`). Every second, query `.ytp-caption-segment` → if text is new, add to buffer. Simple, reliable, guaranteed to work.

Problem 5: Groq Rate Limiting in Live Mode

Issue: Live mode sent API calls too fast (extraction + verification for each batch). Hit 30 RPM limit and 100K TPD limit. **Solution:** Implemented a queue-based processing system. Batches go into a queue, processed sequentially with 2-3 second delays between calls. Verification uses a single API call for all claims in a batch (not one-by-one). Rate limit errors trigger a 30-second wait and retry.

Problem 6: WebSocket Connection Failing from Offscreen Document

Issue: Deepgram WebSocket connection with `['token', API_KEY]` subprotocol returned 400. Tried query parameter `auth` — also 400. The WebSocket silently failed to connect from both service worker and offscreen contexts. **Solution:** The issue was the API key permissions. Created a new Deepgram API key with Admin role (original key only had Member permissions). Also ensured `wss://api.deepgram.com/*` was in `host_permissions` in `manifest.json`. The offscreen document fetches its own token from the backend (same pattern as the reference project).

Problem 7: Tab Audio Muting During Capture

Issue: `chrome.tabCapture` redirects audio away from the tab. When live mode started, the user couldn't hear the video/audio anymore. **Solution:** In the offscreen document's audio pipeline, connected the media stream source to `audioContext.destination` (speakers output). This pipes the captured audio back to the user's speakers while simultaneously processing it for Deepgram: `source.connect(audioContext.destination)` .

Problem 8: Content Script Not Loading on Already-Open Pages

Issue: When installing/reloading the extension, pages that were already open didn't have the content script injected. Clicking "Scan This Page" showed "Could not establish connection." **Solution:** Added `chrome.scripting.executeScript` fallback in the service worker to inject the content script dynamically. Also added clear error messaging telling users to refresh the page.

Problem 9: MBFC Source Scoring Not Displaying

Issue: The MBFC lookup returned the correct data but the side panel wasn't rendering it properly. **Solution:** Fixed the `showResults` function to handle the source object structure correctly and added CSS classes for the different rating levels (HIGH = green, MIXED = amber, LOW = red).

Problem 10: Live History Not Persisting

Issue: When stopping a live session, the results disappeared. No history was saved because the condition checked `liveClaimsData.length > 0` — if no claims were found (rate limit, or no check-worthy claims), nothing was saved. **Solution:** Changed to always save the session if there's transcript text OR claims. Added `liveTranscriptText` variable that accumulates all transcribed text, and includes it in the saved history entry.

Problem 11: Vite Module Preload Polyfill Error

Issue: Cannot read properties of null (reading 'replaceChild') error in the side panel console. **Solution:** This is Vite's module preload polyfill trying to inject a link element into the document head of the side panel. Added `window.addEventListener('unhandledrejection', ...)` to suppress it silently. It's cosmetic and doesn't affect functionality.

Problem 12: Chrome Extension CSP Blocking Inline Scripts

Issue: Added an inline `<script>` tag in `sidepanel.html` for error suppression — Chrome's Content Security Policy blocked it. **Solution:** Removed the inline script and moved the error handler into the module JS file instead.

6. Credibility Scoring Formula

Score (0-100) = Weighted average of claim verdicts
+ Source credibility adjustment
- Image flag penalties

Verdict scores: TRUE=100, MISLEADING=40, UNVERIFIABLE=50, FALSE=0
Confidence mult: HIGH=1.0x, MEDIUM=0.8x, LOW=0.5x
Importance mult: HIGH=1.5x, MEDIUM=1.0x, LOW=0.7x
Source adjust: HIGH=+5, MIXED=-5, LOW=-15, VERY LOW=-25
Image penalty: -10 points per flagged image

7. Project Statistics

- Source files: 29
- Lines of code: ~5,500
- MBFC domains: 3,920
- API providers: 3 (Groq + Gemini + Deepgram)
- Backend endpoints: 1 (Vercel serverless)
- Build time: ~300ms
- Extension size: ~190KB compressed
- Total development time: ~2 days of active coding
- Git commits: 25+

8. What Differentiates This From Existing Solutions

Feature	Free Fact Checker (Reference)	NoLie
Input type	Manual text selection only	Articles + YouTube + Live Audio + Images
Automation	None — user must highlight	Full auto-scan with one click
Video support	None	YouTube transcripts + Deepgram live audio
Image analysis	None	AI-generated/manipulation detection
Source scoring	None	MBFC dataset (3,920 domains)
Overall score	None	Weighted 0-100 credibility score

Heatmap	None	Color-coded inline on article
Custom prompts	Yes	Yes
History	None	Full history with detail view
Export	None	HTML reports with citations
Backend	None	Vercel serverless for Deepgram tokens
Cost	Requires Gemini quota	Mostly free (Groq free tier)
Real-time	None	Live fact-checking while video plays

9. Demo Script for Meeting

1. **Article scan:** Open BBC article → Scan → show claims with verdicts, score, heatmap
2. **YouTube transcript:** Open news video → Show transcript opening → Scan → claims from video
3. **Live mode (YouTube):** Play video with CC on → Live tab → Start → claims appear in real-time
4. **Live mode (Deepgram):** Open Instagram reel → Live tab → Start → see transcript appearing + claims
5. **Manual mode:** Select text → right-click → "Fact Check This"
6. **History:** Show past scans, click one, see full results + transcript
7. **Export:** Download HTML report
8. **Settings:** Show dual API keys, custom prompt, image toggle